

Computer Science & IT



Compiler Design



Parsers

Lecture No. 05



By- Ravindra Sir

Recap of Previous Lecture



Topic

Topic

Topic

Parser
operator precedence parser

Topics to be Covered



Topic

Topic

Topic

OPP

operator precedence parser:-

To construct OPP, the grammar should be operator grammar.

Operator grammar:-

The grammar that does not contain NULL productions and two adjacent non terminals on RHS of any production is called operator grammar.

$$A \rightarrow EX$$

$$B \rightarrow \underline{CD}X$$

$$E \rightarrow \underline{EBE} / id \quad X$$

$$B \rightarrow \underline{+ / - / =}$$

$$\textcircled{E \rightarrow E + E / E - E / E = E / id} \rightarrow \text{ambi'}$$

the resulting grammar is ambiguous grammar
 but still we can construct OPP. OPP doesn't bother
 whether the grammar is ambiguous or not, but the grammar
 must be operator grammar.

OPP is the only parser, that can accept some of the ambiguous grammars.

All other parsers (LL, RD, LR) are constructed using unambiguous grammar.



Convert the following grammar to operator grammar

$$S \rightarrow S \underline{A} S / a$$

$$A \rightarrow \underline{b} S \underline{b} / \underline{b}$$

$$S \rightarrow S b S b A / S b S / a$$

$\textcircled{P} \rightarrow \textcircled{\underline{S}\underline{R}} / \underline{S} \rightarrow \textcircled{1}$
 $\underline{R} \rightarrow \underline{b}\underline{S}\underline{R} / \underline{b}\underline{S} \rightarrow \textcircled{2}$
 $\underline{S} \rightarrow \underline{w}\underline{b}\underline{S} / \underline{w}$
 $\underline{w} \rightarrow \underline{L} * \underline{w} / \underline{L}$
 $\underline{L} \rightarrow \underline{id}$

✓
 $\textcircled{7 \text{ min}}$

$\overset{P}{P} \rightarrow Sb \overset{\checkmark}{\textcircled{\checkmark}{SR}} / SbS / S$
 $R \rightarrow bSR / bS$
 $S \rightarrow wbs / w$
 $L \rightarrow L * w / L$
 $L \rightarrow id$

✓
 operat^d grammar
 =

OPP mainly works by considering the precedence among the operators. That is why it is called operator precedence parser.

Though the grammar is ambiguous, the parser preserves the actual precedence among the operators in a table called precedence relation table.

→ In this table, the actual precedence among the operators are preserved as precedence relation $<$, $>$, $=$

Precedence relations:-



$a > b$ (a "takes precedence over" b), so a is reduced before 'b'

Ex:- $* > +$

$a < b$ (b "takes precedence over" a), so b is reduced before 'a'

$a \doteq b$ (a "has same precedence" as b), so both are reduced at the same time

$E \rightarrow E + E / E * E / id \rightarrow$ ambiguous grammar

Precedence relation table

Left $\rightarrow$ Right

	id	+	*	$\$$
id	-	$\circlearrowright$	$\circlearrowright$	$\circlearrowright$
+	$\circlearrowleft$	$\circlearrowright$	$\circlearrowleft$	$\circlearrowright$
*	$\circlearrowleft$	$\circlearrowright$	$\circlearrowright$	$\circlearrowright$
$\$$	$\circlearrowleft$	$\circlearrowleft$	$\circlearrowleft$	-

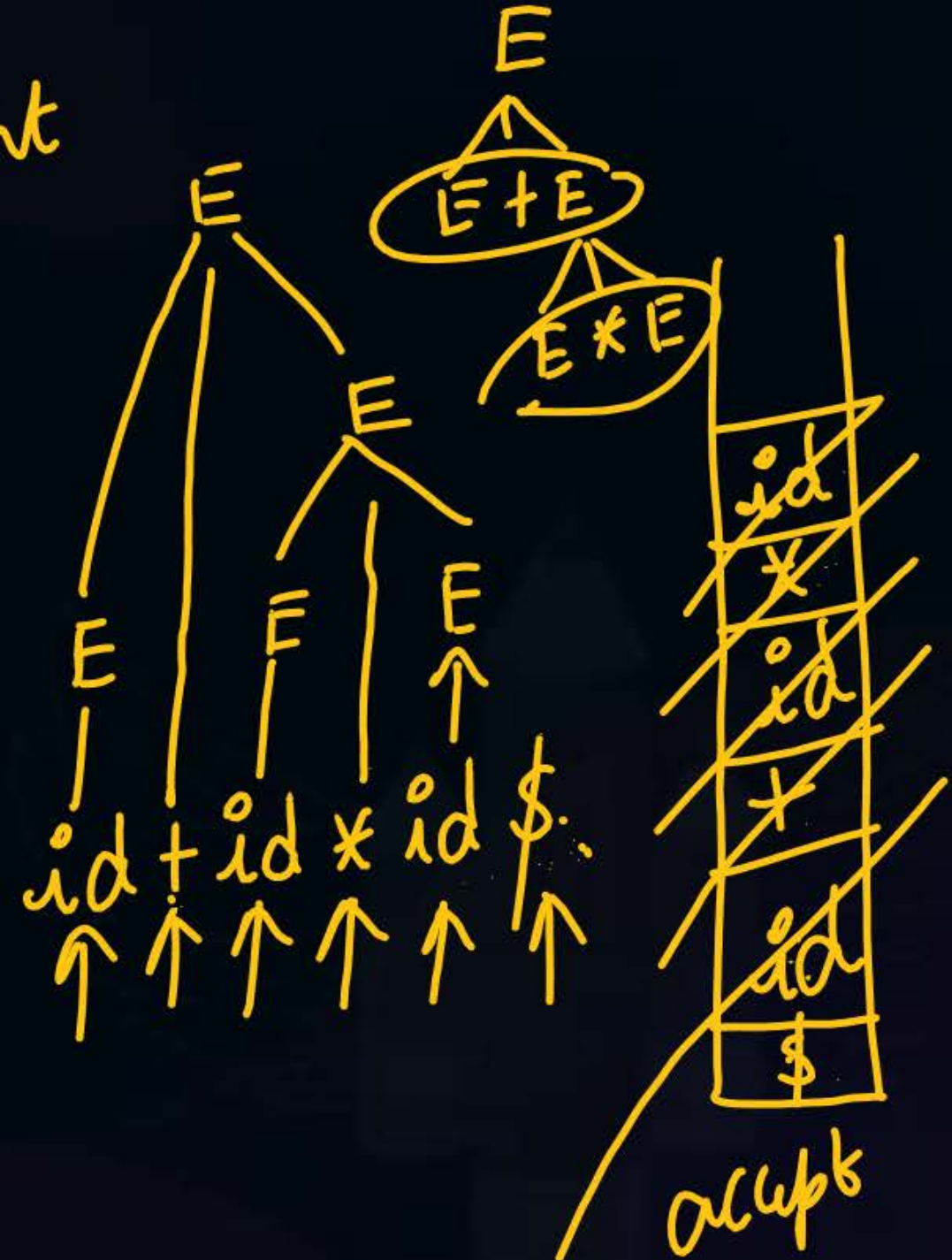
$+ \circlearrowright S$
 $id \circlearrowright +$
 $+ \circlearrowleft id$
 $+ \circlearrowright +$
 $+ \circlearrowleft *$
 $\underline{* \circlearrowright *}$

$E \rightarrow E + E / E * E / id \rightarrow$ ambiguous grammar

Precedence relation table

	id	+	*	\$
id	-	$\circlearrowright$	$\circlearrowright$	$\circlearrowright$
+	$\circlearrowleft$	$\circlearrowright$	$\circlearrowleft$	$\circlearrowright$
*	$\circlearrowleft$	$\circlearrowright$	$\circlearrowright$	$\circlearrowright$
\$	$\circlearrowleft$	$\circlearrowleft$	$\circlearrowleft$	-

Right



$E \rightarrow E + E / E * E / id \rightarrow$ ambiguous grammar

Precedence relation table

	id	+	*	$\$$
id	-	$\circ$	$\circ$	$\circ$
+	$\circ$	$\circ$	$\circ$	$\circ$
*	$\circ$	$\circ$	$\circ$	$\circ$
$\$$	$\circ$	$\circ$	$\circ$	$\circ$

Left → Right



all



Parsing algorithm for operator precedence parser:-



1) Initially, stack contains only $\$$ and i/p buffer contains $w\$$
where w is i/p string

2) Repeat forever
a) let 'a' be the top of the stack and 'b' is the
current element pointed by i/p pointer, that is,
look ahead symbol

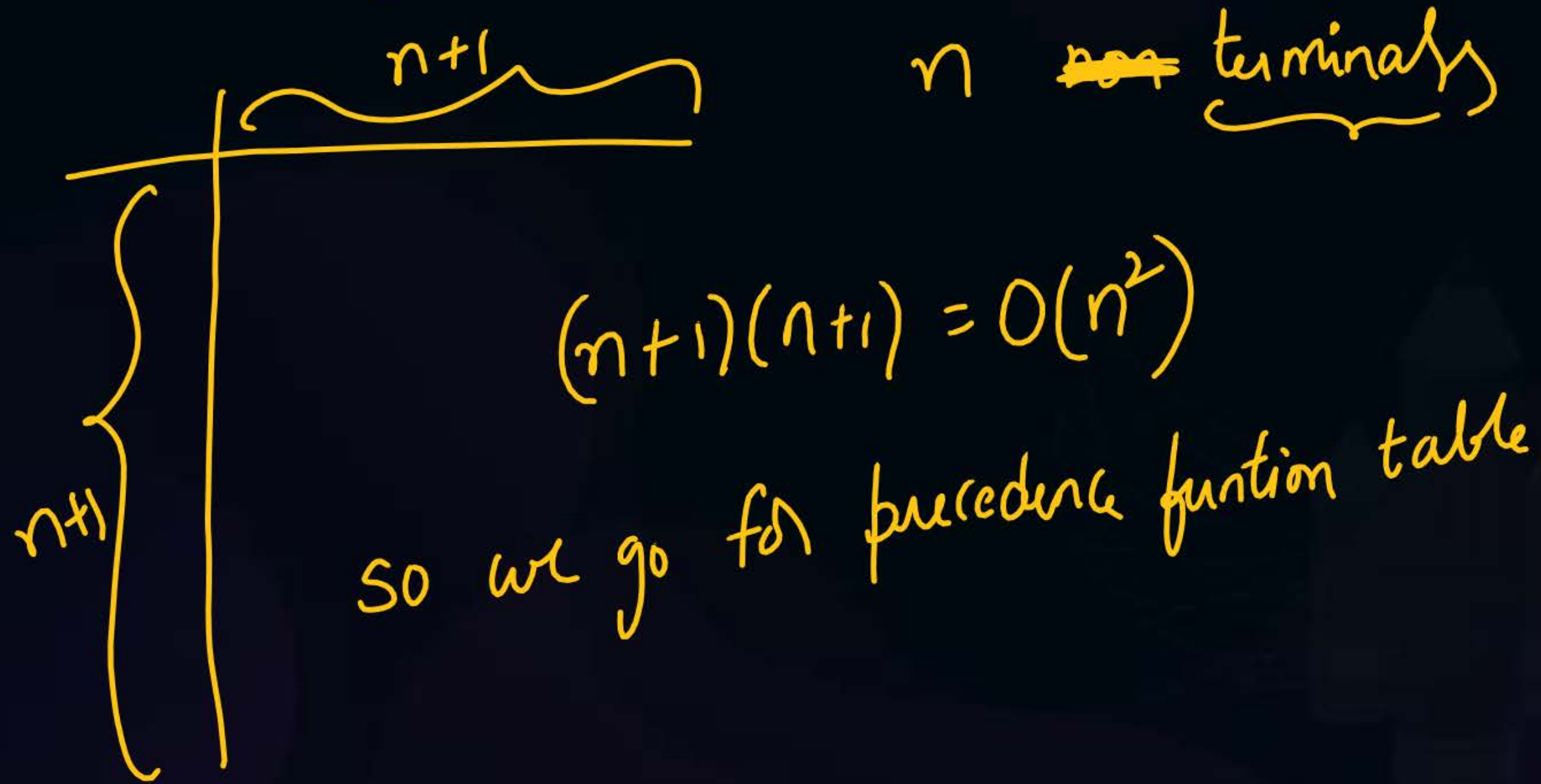


2) b) if $a < b$ $\wedge$ $a \neq b$, push 'b' onto the stack
and increment i/p pointer.

c) if $a > b$ then (Reduce action) repeat
pop the stack until top of the
stack is $< b$ ✓

d) if $a = b = \$$ announce successful completion.

The disadvantage of precedence relation table is that the size is $O(n^2)$.



Procedure to convert precedence relation table to precedence function table.

Input: Precedence relation table

Step 1:- For every terminal 'a' create f_a and g_a

Step 2:- Divide the symbols into groups by using '=' relation

For example:- $a=b$ then f_a and g_b are grouped into one group (f_a, g_b) ✓



2 mins Summary



Topic

Parsers

Topic

Topic

Topic

Topic



THANK - YOU